WEEK 02

Comparisons and branches

This week: learn, practise, build

01 · Python

Comparisons and branches
Understand the idea and trace examples.

02 · Practice

Practise the same Python concept.
Predict, change, diagnose and explain.

03 · Project

Review-first dispatch decision
Build, discuss and test with a peer.

One weekly component moves the final project forward.

Python foundations

Comparisons and branches
Concept → trace a small example → explain its behavior

Values, conditions and complete decisions

A value has a type

False is a Boolean. "False" is text.
= assigns; == compares.

Every path needs a result

Indent each branch. Assign its output.
Rerun after changing an input.

Order comes from the rule

Test the highest-priority condition first.
One chain chooses one result.

A small, stated rule is enough to learn and test branching.

Conditions choose which statements run

Compare values

A comparison returns True or False. >= includes the boundary; > excludes it.

Combine conditions

and needs both conditions; or needs either. not reverses a Boolean.

Choose one branch

if / elif / else takes the first matching block. Indentation defines each block.

Understand the behavior before changing the example.

A condition chooses the next path

1

Input

A Boolean or comparison

2

Branch

True: run this block
False: use the other block

3

Continue

Statements after the branch run for either path

Read the diagram, then trace the same steps in code.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
a >= b	Compare; include equality	Check a boundary
not / and / or	Combine Boolean values	Express a condition
if / elif / else	First matching branch	Choose one outcome
: + indentation	Begin and group a block	Control what runs together

Python Basics · sections 00, 04 · use only when you need a lookup.

Trace one selected branch

```
01 score = 20
02 enabled = True
03 print(score >= 20)
04 if enabled and score >= 20:
05     print("ready")
06 else:
07     print("waiting")
08 print("finished")
```

1. Expected: True, ready, finished.
2. Try score = 19, then enabled = False.
3. The final statement runs after either branch.

Predict → execute → explain the value at each step.

Comparisons produce Boolean values

```
01 total_portions = 20
02 reviewed = True
03 print(reviewed)
04 print(total_portions >= 20)
05 print(total_portions > 20)
06 print(reviewed and total_portions >= 20)
```

1. Predict each Boolean result.
2. >= includes equality; > excludes it.
3. Use Boolean values without quotes.

A comparison evaluates to True or False.

if / else: follow the selected block

```
reviewed = False
if reviewed:
    next_step = "inspect workload"
    print("Review recorded")
else:
    next_step = "coordinator follow-up"
    print("No review recorded")
print(next_step)
assert next_step == "coordinator follow-up"
```

1. Follow the path for False.
2. Both branches assign next_step.
3. Indentation defines each block.

A shared print runs after either branch.

if / elif / else: first matching branch

```
containers = 4
if containers == 0:
    draft_label = "empty"
elif containers < 4:
    draft_label = "small"
else:
    draft_label = "several"
print("Draft:", draft_label)
assert draft_label == "several"
```

1. Which branch handles four containers?
2. Trace zero → empty; three → small
3. Only the first matching branch runs

Order matters: zero matches empty before the small-count comparison.

and / or: compare the truth values

```
01 reviewed = False
02 total_portions = 21
03 print("AND:", reviewed and total_portions >= 20)
04 print("OR:", reviewed or total_portions >= 20)
05 assert (reviewed and total_portions >= 20) is False
06 assert (reviewed or total_portions >= 20) is True
```

1. Predict the two expressions.
2. False and True differs from False or True.
3. Change one input and compare again.

and requires both conditions; or requires at least one.

Boundary testing: > versus >=

```
01 total_portions = 20
02 buggy = total_portions > 20
03 policy = total_portions >= 20
04 print("Buggy:", buggy)
05 print("Policy:", policy)
06 assert buggy is False
07 assert policy is True
```

1. Which comparison agrees with the stated policy?
2. At 19 both comparisons are False
3. At 21 both comparisons are True

Keep the agreed expectation while diagnosing the implementation.

Hands-on practice

Week_02.ipynb · Session 2
Predict → run → change → explain

Your turn: practise the same Python skills

P1 · Compare

For scores 59, 60 and 61, compare ==, > and >=.

P2 · Choose two paths

Write both enabled/disabled paths.
Test each Boolean value.

P3 · Combine decisions

Disabled first; otherwise ready
at 60 or above, retry below 60.

New after Week 1: comparisons, branch priority and boundary tests.

Check your work before asking for help

A complete implementation

Write the whole operation. Do not stop at one changed character.

Independent expected results

Calculate a normal case and an edge case before running your program.

A peer can explain it

Swap driver and reviewer. Explain one line and one test.

Trace comparisons, Boolean expressions and the selected branch.

Project studio

Review-first dispatch decision
Short design briefing → most time for your work

The people’s task this week

1

Someone needs help

The coordinator needs a rule for when a request can proceed.

2

This week, you will

Use the supplied review decision and total to choose hold, standard or team.

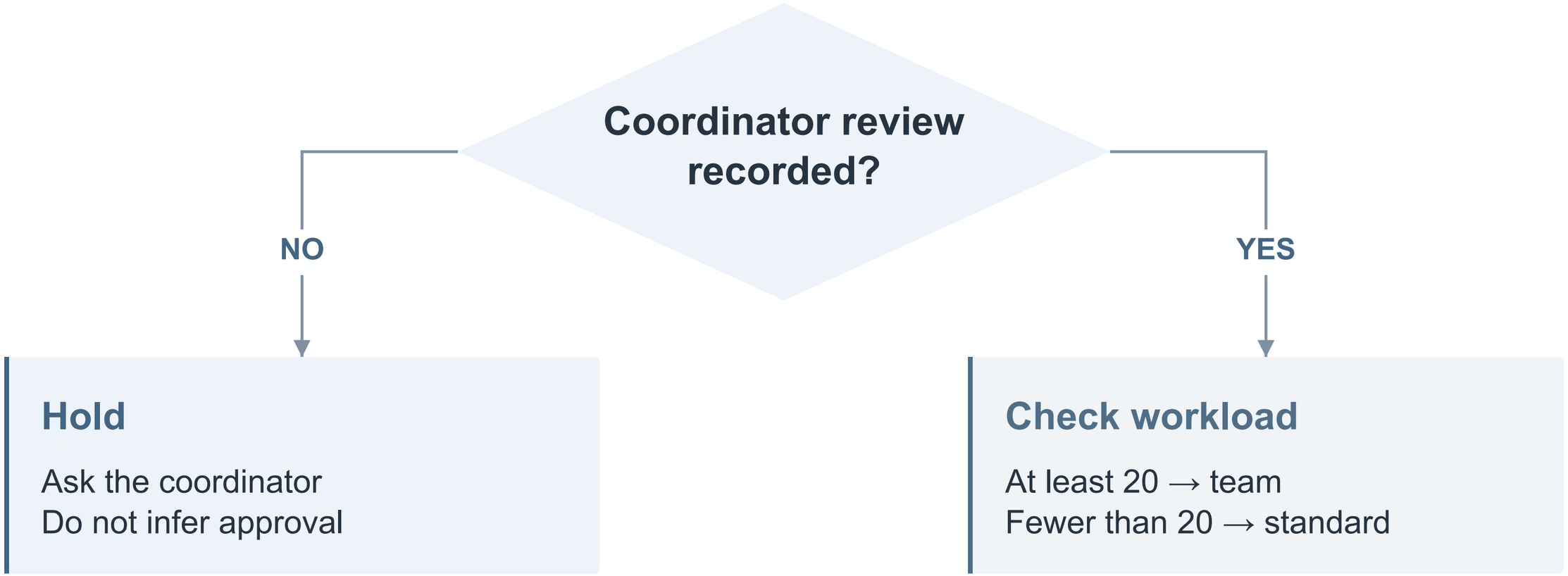
3

A useful result

The coordinator can see the label and the reason for it.

Use the supplied starting files. Focus on one useful result and check it.

Sketch the rule before implementing it



Review comes before the workload decision.

Your implementation TODO

Supplied starting point

The notebook provides test inputs and a branching starter.

You implement / do

Implement review-first branching: unreviewed → hold; reviewed and total >= 20 → team; otherwise standard.

Check the evidence

Test 19, 20 and 21 with both review flags. Unreviewed always stays hold; exactly 20 is team when reviewed.

Unreviewed stays hold; reviewed totals of at least 20 use team.

Show the component working

Step	What to show
1 · Locate	Identify the input, this component and its next user.
2 · Implement	Implement hold / standard / team with review first.
3 · Check	Test 19, 20, 21 with review True and False.
4 · Explain	Show your code, its actual result and why it behaves that way.

Unreviewed stays hold; reviewed totals of at least 20 use team.

Further reading in English

1. ## Python tutorial: if statements

   https://docs.python.org/3/tutorial/controlflow.html#if-statements

2. ## Python Boolean operations

   https://docs.python.org/3/library/stdtypes.html#boolean-operations-and-or-not

Read one relevant section, then bring an example and a question.